

程序报告

学号：

姓名：

一、问题重述

本题要求设计一个能够在黑白棋对弈过程中自动选择落子位置的智能体。在给定当前棋盘状态以及己方棋子颜色的情况下，算法需要从所有合法落子中选择一个最优决策，使得最终对局结果尽可能有利于己方。

从问题本质来看，这是一个典型的博弈决策问题，具有如下特征：

- 对抗性强：双方轮流行动，己方收益与对手策略高度相关；
- 状态空间大：棋盘状态随对局推进呈指数级增长，难以穷举搜索；
- 确定性与随机性并存：规则确定，但对手策略不可预知；
- 终局目标明确：以棋局结束时双方棋子数量作为胜负判据。

二、设计思想

1. 棋盘与规则建模（Board 类设计思想）

题目已经给出黑白棋规则建模，接口为：

`_move`：执行落子操作并返回被翻转棋子的坐标；

`_can_fliped`：判断某一落子是否合法，若合法则返回所有被翻转棋子的坐标列表；

`get_legal_actions`：按照规则生成当前棋手所有合法落子；

`count`：统计某一方棋子数量；

`get_winner`：根据棋子数量判断胜负；

`is_on_board`：判断坐标是否越界；

`display`：输出当前棋盘状态。

通过上述接口，将复杂规则逻辑封装在棋盘类中，为后续搜索算法提供清晰、可靠的状态转移支持。

2. 蒙特卡洛树搜索（MCTS）总体思想

由于黑白棋的状态空间巨大，传统极大极小搜索在有限时间内难以穷举，因此采用蒙特卡洛树搜索作为核心决策算法。MCTS 的基本思想是通过大量随机模拟完整对局，对不同落

子选择的胜负情况进行统计，从而在当前状态下做出近似最优决策。它不依赖人工设计的评估函数，适合规则明确但状态空间庞大的博弈问题。

3. MCTS 四个核心阶段设计——经典 MCTS 算法

(1) 选择 (Selection)

算法从根节点出发，递归选择子节点，直到到达终止节点，或尚未被完全扩展的节点。

选择过程采用 UCB (Upper Confidence Bound) 策略，在“利用已有经验”和“探索未知分支”之间进行平衡，并在过程中记录节点访问次数和节点的奖励统计信息。

(2) 扩展 (Expansion)

若当前节点不是终局节点，且存在尚未被扩展的合法动作，则随机选择一个未扩展的合法落子，生成对应的新子节点，并加入搜索树。

(3) 模拟 (Simulation)

从新扩展的节点出发，采用随机策略进行对局模拟：每一步从当前合法落子中随机选择，直至棋局终止或达到最大模拟步数。模拟阶段的策略与选择阶段不同，其目标是快速获得终局结果，而不是精确决策。

(4) 反向传播 (Back Propagation)

模拟结束后，将终局结果（如黑白棋子数量）沿搜索路径向上回传，更新路径上各节点的访问次数和黑棋/白棋胜利统计信息。在这个过程中，搜索树逐渐向高胜率分支倾斜。

4. UCB 与多臂赌博机问题

UCB 策略来源于多臂赌博机 (Multi-Armed Bandit) 问题，其核心目标是在多种选择收益未知的情况下，最大化长期收益。设每个“臂”的期望收益分别为：(0.3, 0.4, 0.5, 0.6, 0.7)。

若始终采用纯贪心策略（始终选择当前平均收益最大的臂），在有限次数（如 10000 次）实验中，算法可能因早期随机波动而过早陷入次优选择。UCB 通过在平均收益基础上加入探索项，有效避免了这一问题，其思想被直接应用于 MCTS 的节点选择阶段，使算法能够在早期充分探索、后期稳定收敛。

5. 伪代码

输入：当前棋盘状态 board, 当前执棋方 color

输出：最佳落子 action

初始化根节点 root(state = board, color = color)

for t = 1 to 最大模拟次数 do

 node $\leftarrow$ root

 // Selection + Expansion

 while node.state 不是终局 do

 if 当前执棋方无合法落子 then

 break

 if node 未完全展开 then

 随机选择一个未扩展的合法动作 action

 new_state $\leftarrow$ 在 node.state 上执行 action

 new_color $\leftarrow$ 切换执棋方

 node $\leftarrow$ 新建子节点(new_state, new_color, action)

 break

 else

 node $\leftarrow$ 使用 UCB 策略选择 node 的一个子节点

 end if

 end while

 // Simulation

 sim_state $\leftarrow$ node.state 的拷贝

 sim_color $\leftarrow$ node.color

 while sim_state 不是终局 且 未超过最大步数 do

 if sim_color 有合法落子 then

 随机选择合法动作并执行

 end if

 sim_color $\leftarrow$ 切换执棋方

 end while

 // Backpropagation

 统计终局黑棋数 black_score 与白棋数 white_score

 沿搜索路径向上更新：

 visit 次数

 blackwin 与 whitewin

end for

从 root 的子节点中选择 UCB 值最大的节点

返回该节点对应的 action

6. 未来优化方向

(1) 参数层面：UCB 中探索系数 (uct_scalar) 目前固定为 0，可通过调节该参数改善探索与利用的平衡；模拟次数 maxt 可根据时间限制进行自适应调整。

(2) 模拟策略：当前采用纯随机策略，可能导致评估噪声较大；可引入一些启发式规则提升模拟质量。

(3) 终局判定：目前直接以棋子数量作为胜负依据，可将结果归一化或改为胜/负/平三值奖励，增强稳定性。

三、代码内容

```
import copy
import math
import random

class Node:
    def __init__(self, state, color, parent = None, action = None):
        self.visit = 0
        self.blackwin = 0
        self.whitewin = 0
        self.reward = 0.0
        self.state = state
        self.children = []
        self.parent = parent
        self.action = action
        self.color = color

    def add_child(self, new_state, action, color):
        child_node = Node(new_state, parent=self, action = action, color=color)
        self.children.append(child_node)

    def if_fully_expanded(self):
        cnt_max = len(list(self.state.get_legal_actions(self.color)))
        print("cnt_max = ", cnt_max)
        cnt_now = len(self.children)
        print("cnt_now = ", cnt_now)
        if(cnt_max <= cnt_now):
            return True
        else:
            return False
```

```
class AIPlayer:
    """
    AI 玩家
    """

    def __init__(self, color):
        """
        玩家初始化
        :param color: 下棋方, 'X' - 黑棋, 'O' - 白棋
        """

        self.color = color

    def if_terminal(self, state):
        # to see a state is terminal or not
        action_black = list(state.get_legal_actions('X'))
        action_white = list(state.get_legal_actions('O'))
        if(len(action_white) == 0 and len(action_black) == 0):
            return True
        else:
            return False

    def back_propagate(self, node, blackw, whitew):
        while(node is not None):
            node.visit+=1
            node.blackwin+=blackw
            node.whitewin+=whitew
            node = node.parent
        return 0

    def reverse_color(self, color):
        if(color == 'X'):
            return 'O'
        else:
            return 'X'

    def stimulate_policy(self, node):
        board = copy.deepcopy(node.state)
        color = copy.deepcopy(node.color)
        cnt = 0
        while not self.if_terminal(board):
            actions = list(node.state.get_legal_actions(color))
            if(len(actions)==0):
                #no way to go
                color = self.reverse_color(color)
```

```

        else:
            #have ways to go
            action = random.choice(actions)
            board.__move(action,color)
            color = self.reverse_color(color)
        cnt+=1
        if cnt>20:
            break
    return board.count('X'),board.count('O')

def ucb(self,node,uct_scalar=0.0):
    max = -float('inf')
    max_set=[]
    for c in node.children:
        exploit = 0
        if c.color == 'O':
            exploit = c.blackwin/(c.blackwin+c.whitewin)
        else:
            exploit = c.whitewin/(c.blackwin+c.whitewin)
        explore = math.sqrt(2.0*math.log(node.visit)/float(c.visit))
        uct_score = exploit+uct_scalar*explore
        if(uct_score==max):
            max_set.append(c)
        elif(uct_score>max):
            max_set=[c]
            max = uct_score
    if(len(max_set)==0):
        print("max_set is empty")
        print(len(node.children))
        node.state.display()
        return node.parent
    else:
        return random.choice(max_set)

def expand(self,node):
    actions_available = list(node.state.get_legal_actions(node.color))
    actions_already = [c.action for c in node.children]
    if(len(actions_available)==0):
        return node.parent
    action = random.choice(actions_available)
    while action in actions_already:
        action=random.choice(actions_available)
    print(action)

```

```

new_state = copy.deepcopy(node.state)
new_state._move(action,node.color)
new_state.display()
new_color = self.reverse_color(node.color)
node.add_child(new_state,action = action,color= new_color)
return node.children[-1]

def select_policy(self,node):
    while(not self.if_terminal(node.state)):
        if(len(list(node.state.get_legal_actions(node.color)))==0):
            return node;
        elif(not node.if_fully_expanded()):
            print("need to expand")
            new_node = self.expand(node)
            print("end of expand")
            return new_node
        else:
            print("fully expanded")
            node.state.display()
            print(len(node.children))
            print(list(node.state.get_legal_actions(node.color)))
            node = self.uch(node)
    return node

def MCTS_search(self,root,maxt = 100):
    #print("root state :")
    #root.state.display()
    for t in range(maxt):
        print("$$$$$$$$$$$t = ",t)
        leave = self.select_policy(root)
        #print("leave state:")
        #leave.state.display()
        blackwin,whitewin = self.stimulate_policy(leave)
        self.back_propagate(leave,blackw=blackwin,whitew=whitewin)
    #print("root state :")
    #root.state.display()
    return self.uch(root).action

def get_move(self, board):
    """
    根据当前棋盘状态获取最佳落子位置
    :param board: 棋盘
    :return: action 最佳落子位置, e.g. 'A1'
    """

```

```

if self.color == 'X':
    player_name = '黑棋'
else:
    player_name = '白棋'
print("请等一会，对方 {}-{} 正在思考中...".format(player_name, self.color))

# -----请实现你的算法代码-----

action = None
root_board = copy.deepcopy(board)
root = Node(state=root_board,color=self.color)
action = self.MCTS_search(root)

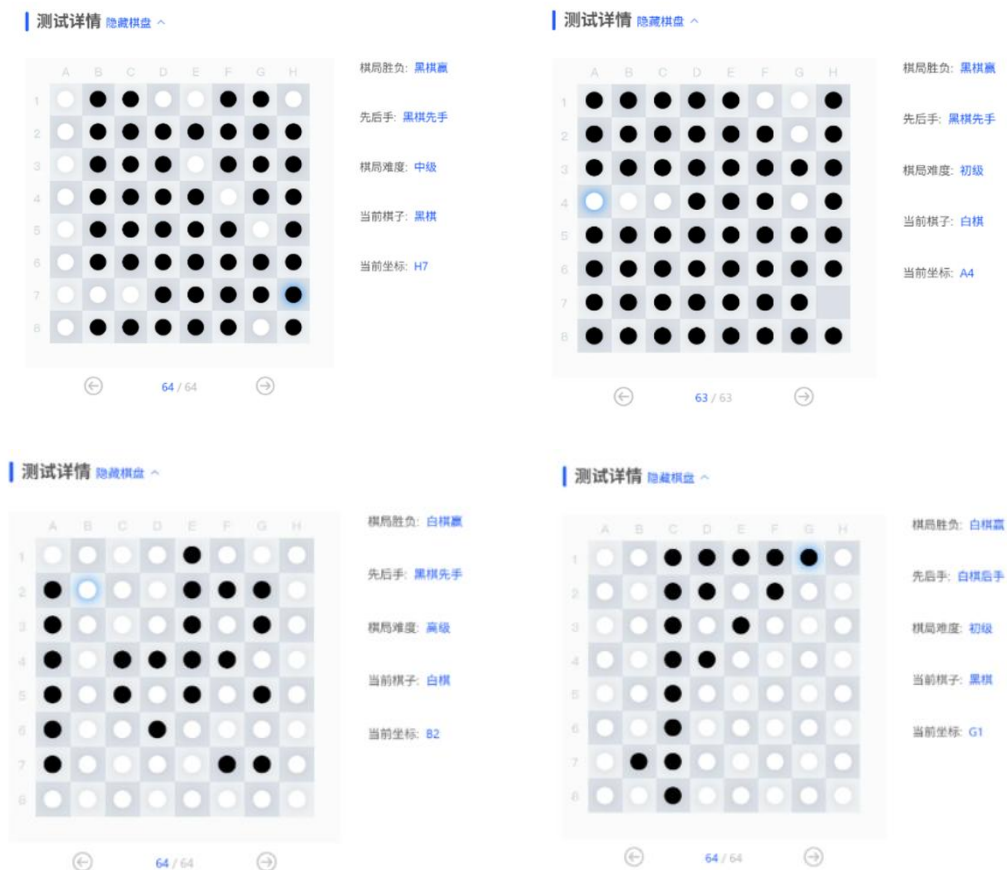
# -----

return action

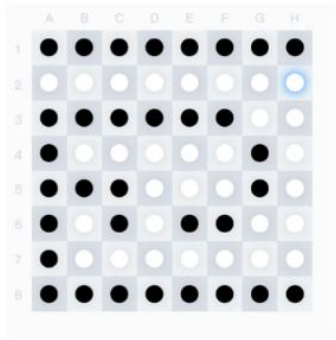
```

四、实验结果

我们使用传统蒙特卡洛树搜索算法设计了一个 AI 玩家。在实验平台的测试中，对于黑/白棋先手，不同难度级别的试验，都取得了较好的效果(胜率 66%，初级难度下的胜率 100%)。如下图所示：



测试详情 隐藏棋盘 ^



棋局胜负: 黑棋赢

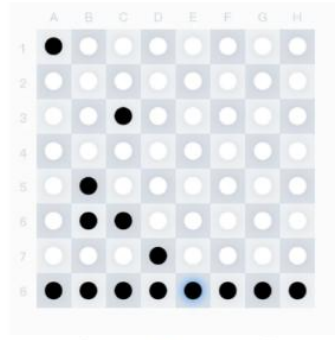
先后手: 白棋后手

棋局难度: 中级

当前棋子: 白棋

当前坐标: H2

测试详情 隐藏棋盘 ^



棋局胜负: 白棋赢

先后手: 白棋后手

棋局难度: 高级

当前棋子: 黑棋

当前坐标: E8

此外，我们对 Game 类结尾进行了简单的修改，设计 AI 与 AI 对弈，得到了这样的效果：

====游戏结束!====

```
A B C D E F G H
1 X X X X X X 0 X
2 X X X X X 0 0 X
3 X X X X 0 X 0 X
4 X X X 0 X X 0 X
5 X X X X 0 X 0 X
6 X X X 0 0 X 0 X
7 X 0 X 0 0 0 X X
8 X X X X X X X X
```

统计棋局： 棋子总数 / 每一步耗时 / 总时间

黑 棋： 48 / 0 / 0

白 棋： 16 / 0 / 0

黑棋获胜！

我们参考老师给出的例程进行了人机对弈，效果如下：

可以看到，在六分钟之内，我们的同学就被 AI 玩家杀得落花流水。

end of expand

```
A B C D E F G H
1 . . . . X 0 0
2 . . . . 0 0 0 0
3 . 0 0 0 0 0 0 0
4 . . 0 0 0 X 0 0
5 . X X 0 X 0 0 0
6 0 X 0 0 0 0 0 0
7 . X X . 0 0 0 0
8 . X . . 0 . 0
```

统计棋局： 棋子总数 / 每一步耗时 / 总时间

黑 棋： 9 / 5 / 345

白 棋： 35 / 0 / 0

请'黑棋-X'方输入一个合法的坐标(e.g. 'D3', 若不想进行, 请务必输入'Q'结束游戏.): |

五、总结

在这个作业中,我们围绕黑白棋博弈问题,设计并实现了一个基于蒙特卡洛树搜索的 AI 玩家。实验目标是在给定棋盘状态和执棋方颜色的条件下,使 AI 能够在有限时间内自动选择合理落子,并在对弈中取得较好的胜率。

从测试结果来看,程序基本达到了预期目标。程序完整实现了 MCTS 的四个核心阶段——选择、扩展、模拟和反向传播。我们设计的 AI 玩家能够正确理解黑白棋规则,在不同先后手和难度条件下都能够做出较为优秀的决策。在与 AI 玩家以及人类玩家的对弈测试中,我们的 AI 玩家都有着明显的优势,在低难度和人机对弈场景下胜率很高,这也验证了蒙特卡洛树搜索算法在该问题上的有效性。

同时,也有一些不足与改进空间。首先,当前实现采用的是标准 MCTS 算法的基础版本,未引入启发式评估或改进型搜索策略;其次,在模拟阶段采用纯随机策略,可能导致估值噪声较大,在复杂局面下搜索效率不高;此外,UCB 公式中的探索系数固定为 0,实际运行中更偏向于贪心选择,限制了算法对未知分支的探索能力。